

Trò chơi Ulam và mã hóa

Yu Wei

Nguồn gốc: Ulam's game and coding

IOI China candidate team papers / 2002 / Yu Wei.pdf

1 Trò chơi Ulam

Trong trò chơi Ulam có n số. Trong số đó có đúng một số khác với các số còn lại; đó là số mà người chơi đã chọn. Ta được đặt các câu hỏi dạng “số ấy có nằm trong tập X hay không?” và mỗi câu hỏi chỉ nhận được câu trả lời “có” hoặc “không”. Khó khăn nằm ở chỗ trong toàn bộ quá trình có thể xuất hiện một câu trả lời sai, nhưng số lần sai không vượt quá một.

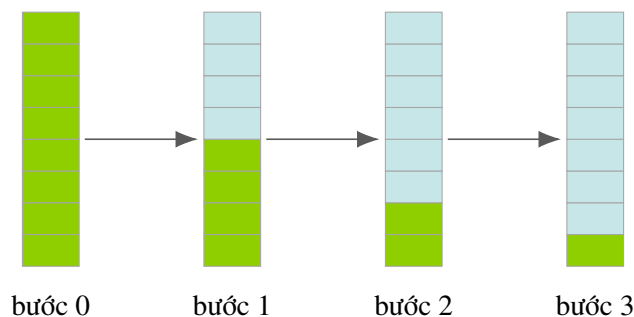
Mục tiêu là tìm ra số đã được chọn bằng càng ít câu hỏi càng tốt.

2 Bài toán gốc không có sai sót

Trước hết xét phiên bản đơn giản: vẫn có n số và một số được chọn, cách hỏi vẫn là hỏi số được chọn có thuộc một tập X hay không, nhưng mọi câu trả lời đều đúng.

Lời giải quen thuộc là **binary search**. Mỗi lần ta chia tập các số còn có thể là đáp án thành hai phần có kích thước gần bằng nhau nhất, lấy một phần làm tập X , rồi dựa vào câu trả lời để loại bỏ phần không thể chứa đáp án. Sau khoảng $\lg n$ câu hỏi chỉ còn lại một số, và số đó là đáp án.

Nhìn theo một cách khác, binary search chỉ là một phương pháp sàng lọc hơi tham lam: ở mỗi bước nó cố gắng loại bỏ nhiều số nhất trong trường hợp xấu nhất.



Sơ đồ trên chỉ biểu diễn ý tưởng sàng lọc: phần xanh là tập còn có thể chứa đáp án, và sau mỗi câu hỏi ta giữ lại phần phù hợp với câu trả lời.

3 Binary search khi có một câu trả lời sai

Khi có thể có một câu trả lời sai, việc chia đôi theo số lượng không còn đủ. Nguyên tắc đúng hơn là chia đều *khả năng* chứa số được chọn: hai phần sau khi hỏi phải có xác suất chứa đáp án như nhau, xét theo

mọi kịch bản sai sót còn hợp lệ.

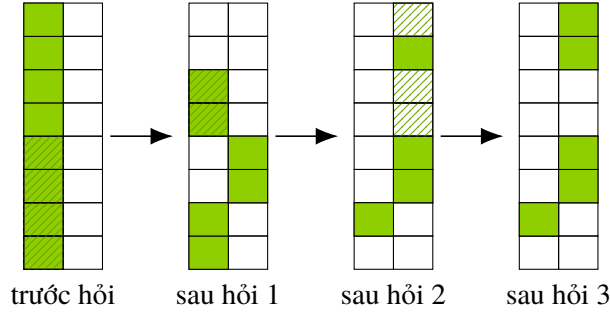
Một cách hiện thực là phân loại các số theo số lần câu trả lời trước đó phải sai thì số ấy mới còn có thể là đáp án. Với giả thiết nhiều nhất một lỗi, ta có hai lớp:

- lớp 0: các số còn phù hợp nếu chưa có câu trả lời sai nào;
- lớp 1: các số chỉ còn phù hợp nếu đã có đúng một câu trả lời sai.

Trong cùng một lớp, các số có cùng mức khả dĩ. Vì vậy ở mỗi câu hỏi ta cố gắng chia đều từng lớp, lấy một phần của mỗi lớp đưa vào tập X .

3.1 Một thao tác mẫu

Hình sau mô phỏng bốn trạng thái liên tiếp. Mỗi khối gồm hai cột: cột trái là lớp 0, cột phải là lớp 1. Các ô xanh là những số còn hợp lệ trong lớp tương ứng; phần gạch chéo là phần bị xác định là không chứa số được chọn sau câu trả lời hiện tại và sẽ chuyển sang lớp lỗi cao hơn hoặc bị loại.



Điểm quan trọng của hình không phải là vị trí cụ thể của từng ô, mà là cách mỗi câu hỏi chia đều từng lớp lỗi rồi cập nhật lớp của các số theo câu trả lời.

3.2 Ước lượng số câu hỏi

Giả sử dùng cách sàng lọc trên. Sau k câu hỏi, số phần tử còn trong lớp 0 là

$$\frac{n}{2^k}.$$

Gọi $f(k)$ là số phần tử còn trong lớp 1. Mỗi bước giữ lại một nửa lớp 1 cũ, đồng thời một nửa phần không khớp của lớp 0 chuyển sang lớp 1. Do đó

$$f(k) = \frac{f(k-1)}{2} + \frac{n}{2^k}.$$

Từ truy hồi này suy ra

$$f(k) = \frac{kn}{2^k}.$$

Khi tổng số phần tử ở hai lớp không vượt quá 1, đáp án đã được xác định:

$$f(k) + \frac{n}{2^k} \leq 1.$$

Thay công thức của $f(k)$, ta được điều kiện

$$2^k \geq kn + n.$$

Vì vậy số câu hỏi của phương pháp sàng lọc là nghiệm nguyên nhỏ nhất k thỏa bất đẳng thức trên.

4 Phương pháp mới: mã hóa

Phương pháp sàng lọc có ba điểm bất tiện. Nó phải lưu trạng thái trung gian, phụ thuộc trực tiếp vào kết quả của bước trước, và cần xử lý cẩn thận các trường hợp biên.

Ý tưởng mã hóa tránh các bất tiện ấy bằng cách gán trước cho mỗi số một mã nhị phân cố định. Câu hỏi thứ i sẽ hỏi tập các số có bit thứ i của mã bằng 1. Câu trả lời được đổi thành mã kết quả theo quy ước

$$Y \leftrightarrow 1, \quad N \leftrightarrow 0.$$

Nếu không có lỗi, đáp án chính là số có mã trùng với mã kết quả.

4.1 Ví dụ mã hóa

Với 8 số, dùng ba bit thông tin là đủ. Gán mã theo bảng sau:

số	X_1	X_2	X_3
1	0	0	0
2	0	0	1
3	0	1	0
4	0	1	1
5	1	0	0
6	1	0	1
7	1	1	0
8	1	1	1

Nếu ba câu trả lời là Y, Y, N, mã kết quả là 110. Khi đó hàng tương ứng với số 7 được đánh dấu ở cả ba cột phù hợp, nên số 7 là đáp án.

	X_1	X_2	X_3
1			
2			
3			
4			
5			
6			
7			
8			

110

hàng 7 trùng mã kết quả

5 Lỗi và mã sửa lỗi

Khi trong các câu trả lời có một lỗi, mã kết quả nhận được cũng sai đúng một bit. Vì thế các khả năng trả lời khác nhau có thể được xem như các mã lỗi khác nhau. Ta cần thiết kế **mã sửa lỗi** sao cho từ mã kết quả bị sai một bit vẫn suy ra được mã đúng.

Một cách tự nhiên là dùng mã kiểm tra chẵn lẻ. Các bit kiểm tra được định nghĩa là tổng nhị phân, tức phép xor, của một số bit thông tin. Khi một bit bị sai, mọi đẳng thức kiểm tra liên quan đến bit ấy sẽ không còn đúng.

Giả sử tạm thời các bit không phải bit kiểm tra đều đúng. Ta xét những bit kiểm tra bị “sai”, tức những phương trình kiểm tra không thỏa:

- nếu không có phương trình nào sai, thì không có lỗi;
- nếu chỉ có một phương trình sai, lỗi nằm ở chính bit kiểm tra ấy;
- nếu có nhiều phương trình sai, lỗi nằm ở bit thông tin xuất hiện đúng trong các phương trình kiểm tra ấy.

5.1 Cấu tạo bit kiểm tra

Giả sử có $m = 4$ bit thông tin X_1, X_2, X_3, X_4 . Ta cần thêm r bit kiểm tra sao cho mỗi vị trí bit, cùng với trường hợp không có lỗi, có một dấu hiệu phân biệt. Vì vậy phải có

$$2^r \geq m + r + 1.$$

Với $m = 4$, giá trị nhỏ nhất là $r = 3$. Đặt ba bit kiểm tra là X_5, X_6, X_7 và chọn các phương trình

$$X_5 = X_2 \oplus X_3 \oplus X_4,$$

$$X_6 = X_1 \oplus X_3 \oplus X_4,$$

$$X_7 = X_1 \oplus X_2 \oplus X_4.$$

Mỗi bit thông tin xuất hiện trong một tập phương trình khác nhau:

	X_5	X_6	X_7
X_1	0	1	1
X_2	1	0	1
X_3	1	1	0
X_4	1	1	1

Các cột riêng 100, 010, 001 lần lượt nhận diện lỗi ở X_5, X_6, X_7 , còn 000 biểu thị không có lỗi. Như vậy mọi lỗi một bit đều có một dấu hiệu riêng.

5.2 Đối ứng với câu hỏi

Phép xor trên mã tương đương với phép xor trên chính các câu hỏi. Chẳng hạn từ

$$X_5 = X_2 \oplus X_3 \oplus X_4$$

ta hiểu rằng câu hỏi thứ 5 hỏi tập đối xứng sai khác của ba tập trong các câu hỏi 2, 3, 4. Một số thuộc X_5 khi và chỉ khi nó thuộc lẻ số trong ba tập X_2, X_3, X_4 .

Vì vậy việc thêm bit kiểm tra vào mã không chỉ là thao tác trên ký hiệu nhị phân; nó còn cho ta cách tạo thêm các câu hỏi kiểm tra trong trò chơi Ulam.

6 Số câu hỏi

Kết quả đếm câu hỏi của phương pháp sàng lọc đã thu được ở trên: số câu hỏi là nghiệm nguyên nhỏ nhất k thỏa

$$2^k \geq kn + n.$$

Với phương pháp mã hóa, đặt $m = \lg n$ là số bit thông tin cần có khi không có lỗi, và đặt t là tổng số câu hỏi sau khi thêm bit kiểm tra. Ngoài m bit thông tin, mỗi khả năng lỗi phải ứng với một mã kiểm tra riêng. Do đó cần

$$2^{t-m} \geq t + 1.$$

Mặt khác, tất cả khả năng gồm giá trị thật và vị trí lỗi cũng phải có mã khác nhau, nên điều kiện tương đương là

$$2^t \geq 2^m(t+1).$$

Kết quả là t là nghiệm nguyên nhỏ nhất thỏa

$$2^{t-m} > t + 1.$$

Hai công thức trùng nhau khi $m = \lg n$:

$$2^t \geq tn + n \iff 2^t \geq t2^m + 2^m \iff 2^{t-m} \geq t + 1.$$

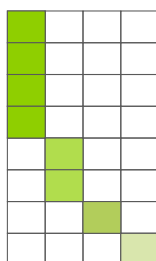
Nói cách khác, theo số câu hỏi trong trường hợp một lỗi, cách sàng lọc và cách mã hóa đạt cùng ngưỡng.

7 Mở rộng

7.1 Nhiều lỗi hơn

Khi số câu trả lời sai có thể lớn hơn một, việc mở rộng trực tiếp phương pháp mã hóa trở nên khó hơn nhiều, vì mã phải sửa được nhiều lỗi. Với hướng sàng lọc, ý tưởng lại khá trực tiếp: nếu có nhiều nhất e lỗi, ta chia các số thành $e + 1$ lớp, trong đó lớp i gồm các số còn hợp lệ nếu đã có đúng i câu trả lời sai. Mỗi lần hỏi, ta cố gắng chia đều từng lớp.

lópłóplóplóp 3



mỗi câu hỏi chia từng lớp lỗi

7.2 Nhiều loại câu trả lời và bài toán cân bi

Nếu mỗi câu hỏi có nhiều hơn hai loại trả lời, phương pháp sàng lọc phải lưu nhiều nhóm trạng thái hơn. Còn với phương pháp mã hóa, ta thay mã nhị phân bằng mã trong cơ số lớn hơn và thiết kế mã sửa lỗi tương ứng.

Bài toán cân bi là ví dụ điển hình. Mỗi lần cân có ba kết quả: đĩa trái nặng hơn, cân bằng, hoặc đĩa phải nặng hơn. Do đó biểu diễn tự nhiên là mã tam phân. Ngoài giá trị của viên bi đặc biệt, kiểu sai lệch cũng cần được mã hóa, rồi ta thiết kế mã sửa lỗi tam phân để nhân viên lỗi.

8 Kết luận

Trò chơi Ulam cho thấy cùng một bài toán hỏi đáp có thể được nhìn theo hai cách bổ sung nhau. Cách sàng lọc quản lý trực tiếp tập các ứng viên và các lớp lỗi; nó mở rộng tự nhiên sang nhiều lớp bằng cách thêm lớp. Cách mã hóa gán trước biểu diễn cho từng số; trong trường hợp một lỗi, mã kiểm tra chắn lẻ cho phép xác định vị trí lỗi và đặt cùng số câu hỏi tối ưu như phân tích sàng lọc.

Các hướng mở rộng chính là sàng lọc khi có nhiều lỗi, lựa chọn biểu diễn mã, thiết kế mã sửa lỗi, và các hệ mã nhiều cơ sở cho bài toán có nhiều loại câu trả lời.